

وقتی به شرط (switch یا if/else زنجیره‌ای) بر اساس **Type Code** رفتارهای متفاوتی اجرا می‌کنه، برای هر نوع به زیرکلاس بساز و رفتار متفاوت رو به متدهای چندریخت (**Polymorphic**) منتقل کن.

تفاوت با Subclasses و State/Strategy

تکنیک	کاربرد
Replace Type Code with Subclasses	وقتی فقط Type Code داری و می‌خواهی Subclass بسازی
Replace Type Code with State/Strategy	وقتی Type Code در زمان اجرا تغییر می‌کنه
Replace Conditional with Polymorphism	درمان مستقیم if / switch های پراکنده - نتیجه نهایی همون Subclasses یا State هست

این تکنیک در واقع همون مفهوم قبلی رو با تمرکز روی حذف شرطها انجام میده.

مثال قبل از ریفکتور

csharp

```

public class Bird
{
    public const int EUROPEAN = 0
    public const int AFRICAN = 1
    public const int NORWEGIAN_BLUE = 2

    public int Type { get; set; }

    public double GetSpeed()
    {
        switch (Type)
        {
            case EUROPEAN: return GetBaseSpeed();
            case AFRICAN: return GetBaseSpeed() - GetLoadFactor() * 10;
            case NORWEGIAN_BLUE: return IsNailed ? 0 : GetBaseSpeed() * 20;
            default: throw new Exception("نوع نامعتبر");
        }
    }
}

```

```

{

;private double GetBaseSpeed() => 50
;private double GetLoadFactor() => 2
    public bool IsNailed { get; set; }

}

```

Replace Conditional with Polymorphism بعد از

csharp

```

// کلاس والد (abstract)
public abstract class Bird
{
    ;()public abstract double GetSpeed
    ;protected double GetBaseSpeed() => 50
}

// هر زیرکلاس منطق خودش
public class European : Bird
{
    ;()public override double GetSpeed() => GetBaseSpeed
}

public class African : Bird
{
    ;public double LoadFactor { get; set; } = 2
    ;public override double GetSpeed() => GetBaseSpeed() - LoadFactor * 10
}

public class NorwegianBlue : Bird
{
    public bool IsNailed { get; set; }
    ;public override double GetSpeed() => IsNailed ? 0 : GetBaseSpeed() * 20
}

```

مزیت

توضیح

حذف کامل switch	هر زیرکلاس منطق خودش رو داره
اضافه کردن نوع جدید آسونه	یه زیرکلاس جدید، بدون دست زدن به بقیه
Open/Closed Principle	کد برای اضافه شدن باز، برای تغییر بسته
منطق متمرکز	منطق هر نوع توی کلاس خودش، پراکنده نیست

⚠ When Not to Use (از صفحه)

- اگه تعداد کمی شرط داری که تغییر هم نمی‌کنن، ساختن زیرکلاس پیچیدگی اضافی ایجاد می‌کنه.
- اگه Type Code در زمان اجرا تغییر می‌کنه، باید از State/Strategy استفاده کنی.

تمرین

این کد رو با **Replace Conditional with Polymorphism** ریفتور کن:

csharp

```
public class Employee
{
    ;public const int REGULAR = 0
    ;public const int MANAGER = 1
    ;public const int EXECUTIVE = 2

    public int Type { get; set; }
    public string Name { get; set; }
    public decimal BaseSalary { get; set; }

    ()public decimal CalculateSalary
    {
        switch (Type)
        {
            ;case REGULAR: return BaseSalary
            ;case MANAGER: return BaseSalary * 1.5m
```

```

;case EXECUTIVE: return BaseSalary * 2m + 5000000
;("نوع نامعتبر")default: throw new Exception
    {
        {

        ()public int GetVacationDays
        }

        switch (Type)
        {
            ;case REGULAR: return 15
            ;case MANAGER: return 20
            ;case EXECUTIVE: return 30
            ;("نوع نامعتبر")default: throw new Exception
        }
        {
            {

```

راهنمایی:

- abstract → Employee
- ExecutiveEmployee , ManagerEmployee , RegularEmployee
- abstract → GetVacationDays و CalculateSalary

بفرست تا بررسی کنیم! 🤖